# Jobsheet 7
# Praktek Dasar Pemrograman

| | |
|---|---|
| **Nama** | Fadel Abe Johansyah |
| **Kelas / No** | TI-1C / 11 |
| **NIM** | 264107020198 |

## Percobaan 1: Studi Kasus Nilai Mahasiswa di SIAKAD – Perulangan FOR

### Kode Program (SiakadFor_11.java):

```
import java.util.Scanner;

public class SiakadFor_11 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double nilai;
        double tertinggi = 0;
        double terendah = 100;

        for (int i = 1; i <= 10; i++){
            System.out.printf("Masukkan nilai mahasiswa ke %d: ",i);
            nilai = sc.nextDouble();
            if (nilai > tertinggi) {
                tertinggi = nilai;
            } if (nilai < terendah){
                terendah = nilai;
            }
        }

        System.out.println("Nilai tertinggi adalah: "+tertinggi);
        System.out.println("Nilai terendah adalah: "+terendah);

        sc.close();
    }
}
```

**Pertanyaan & Jawaban Percobaan 1:**

**1. Sebutkan dan tunjukkan masing-masing komponen perulangan FOR pada kode program Percobaan 1!**

- Inisialisasi: int i = 1 (Menentukan nilai awal variabel counter i).
- Kondisi / Batas Perulangan: i <= 10 (Syarat agar perulangan terus berjalan selama i kurang dari atau sama dengan 10).
- Pengubah Nilai (Update / Increment): i++ (Menambahkan nilai counter i sebanyak 1 pada setiap akhir iterasi).
- Body Perulangan: Blok kode di dalam kurung kurawal { ... } yang diulang untuk menginput nilai, mengecek nilai tertinggi & terendah, serta menghitung jumlah mahasiswa yang lulus/tidak lulus.

**2. Mengapa variabel tertinggi diinisialisasi 0 dan terendah diinisialisasi 100? Apa yang terjadi jika variabel tertinggi diinisialisasi 100 dan terendah diinisialisasi 0?**

- Variabel tertinggi diinisialisasi 0 agar nilai masukan pertama yang bernilai lebih dari 0 dapat langsung menggantikan nilai tertinggi.
- Variabel terendah diinisialisasi 100 agar nilai masukan pertama yang bernilai kurang dari 100 dapat langsung menggantikan nilai terendah.
- Jika dibalik (tertinggi = 100 dan terendah = 0), maka logika penyeleksian akan gagal. Nilai tertinggi tidak akan pernah berubah dari 100 (karena tidak ada nilai input yang > 100) dan nilai terendah tidak akan pernah berubah dari 0 (karena tidak ada nilai input yang < 0).

**3. Jelaskan fungsi dan alur kerja dari potongan kode penyeleksian nilai tertinggi dan terendah!**

- Fungsi: Membandingkan nilai masukan mahasiswa dengan nilai tertinggi dan terendah yang tercatat saat ini.
- Alur Kerja:
  1. Program mengecek kondisi 'if (nilai > tertinggi)'. Jika benar, nilai tertinggi diperbarui dengan nilai baru.
  2. Program mengecek secara terpisah kondisi 'if (nilai < terendah)'. Jika benar, nilai terendah diperbarui dengan nilai baru.
  3. Proses ini diulang selama 10 kali iterasi sehingga di akhir perulangan didapatkan nilai puncak tertinggi dan terendah.

**4. Modifikasi kode program sehingga terdapat perhitungan untuk menentukan berapa mahasiswa yang lulus dan yang tidak lulus berdasarkan batas kelulusan (nilai minimal 60). Tampilkan jumlah mahasiswa lulus dan tidak lulus setelah menampilkan nilai tertinggi dan terendah!**

- Penjelasan Modifikasi: Menambahkan variabel 'lulus' dan 'tidakLulus' berjenis integer diisikan nilai awal 0. Di dalam perulangan for, ditambahkan penyeleksian kondisi 'if (nilai >= 60) lulus++; else tidakLulus++;'. Di luar perulangan, hasil pencacahan lulus dan tidak lulus ditampilkan ke layar.

## Percobaan 2: Studi Kasus Nilai Mahasiswa di SIAKAD – Perulangan WHILE

### Kode Program (SiakadWhile_11.java):

```
import java.util.Scanner;

public class SiakadWhile_11 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int nilai;
        int jml;
        int i = 0;

        System.out.print("Masukkan jumlah mahasiswa: ");
        jml = sc.nextInt();

        while (i < jml) {
            System.out.print("Masukkan nilai mahasiswa ke-" + (i+1) + ": ");
            nilai = sc.nextInt();

            if (nilai < 0 || nilai > 100) {
                System.out.println("Nilai tidak valid. Masukkan lagi nilai yang valid");
                continue;
            }

            if (nilai > 80 && nilai <=100) {
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah A");
            } else if (nilai > 73 && nilai <= 80){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah B+");
            } else if (nilai > 65 && nilai <= 73){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah B");
            } else if (nilai > 60 && nilai <= 65){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah C+");
            } else if (nilai > 50 && nilai <= 60){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah C");
            } else if (nilai > 39 && nilai <= 50){
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah D");
            } else {
                System.out.println("Nilai mahasiswa ke-" + (i+1) + " adalah E");
```

```
            }
            i++;
        }
        sc.close();
    }
}
```

**Pertanyaan & Jawaban Percobaan 2:**

**1. Tentukan maksud dan kegunaan dari sintaks 'nilai < 0 || nilai > 100' dan 'continue'!**

- 'nilai < 0 || nilai > 100': Digunakan sebagai kondisi validasi input. Menandakan jika nilai kurang dari 0 ATAU lebih dari 100 (berada di luar rentang nilai valid 0-100).
- 'continue': Berfungsi untuk menghentikan sisa perintah pada iterasi perulangan saat ini dan langsung melompat kembali ke pemeriksaan kondisi WHILE tanpa mengeksekusi perintah di bawahnya (sehingga nilai tidak valid diabaikan dan i++ tidak dieksekusi).

**2. Mengapa sintaks i++ dituliskan di akhir perulangan WHILE? Apa yang terjadi jika posisinya dituliskan di awal perulangan WHILE?**

- Ditulis di akhir agar penambahan counter i hanya terjadi apabila input nilai mahasiswa sudah dipastikan valid dan selesai diproses kriteria kategorinya.
- Jika ditulis di awal perulangan, variabel i akan langsung bertambah sebelum input dimasukkan. Hal ini menyebabkan indeks mahasiswa bergeser (mulai dari mahasiswa ke-2) dan perulangan akan berhenti lebih awal. Selain itu, jika terjadi continue, i sudah terlanjur bertambah sehingga data mahasiswa tersebut terlewatkan.

**3. Apabila jumlah mahasiswa yang dimasukkan adalah 19, berapa kali perulangan WHILE akan berjalan?**

- Perulangan WHILE akan berjalan sebanyak 19 kali iterasi sukses (dengan nilai i dari 0 hingga 18). Jika pengguna memasukkan nilai tidak valid, total iterasi loop bertambah sesuai banyaknya input salah yang diulang.

**4. Modifikasi kode program sehingga apabila terdapat mahasiswa yang mendapat nilai A, program menampilkan pesan tambahan 'Bagus, pertahankan nilainya'!**

- Penjelasan Modifikasi: Ditambahkan perintah 'System.out.println("Bagus, pertahankan nilainya!");' di dalam blok kondisi 'if (nilai > 80 && nilai <= 100)'.

## Percobaan 3: Studi Kasus Transaksi di Kafe – Perulangan DO-WHILE

### Kode Program (KafeDoWhile_11.java):

```
import java.util.Scanner;

public class KafeDoWhile_11 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int kopi;
        int teh;
        int roti;
        int hargaKopi = 12000;
        int hargaTeh = 7000;
        int hargaRoti = 20000;
        int totalHarga;
        String namaPelanggan;

        do{
            System.out.print("Masukkan nama pelanggan (ketik 'batal' untuk keluar): ");
            namaPelanggan = sc.nextLine();
            if (namaPelanggan.equalsIgnoreCase("batal")) {
                System.out.print("Transaksi dibatalkan.");
                break;
            }

            System.out.print("Jumlah Kopi: ");
            kopi = sc.nextInt();
            System.out.println("Jumlah Teh: ");
            teh = sc.nextInt();
            System.out.print("Jumlah Roti: ");
            roti = sc.nextInt();

            totalHarga = (kopi * hargaKopi) + (teh * hargaTeh) + (roti * hargaRoti);
            System.out.println("Total yang harus dibayar: Rp "+totalHarga);
            sc.nextLine();

        } while(true);
```

```
        System.out.println("Semua transaksi selesai.");
        sc.close();
    }
}
```

**Pertanyaan & Jawaban Percobaan 3:**

**1. Pada penggunaan DO-WHILE ini, apabila nama pelanggan yang dimasukkan pertama kali adalah 'batal', maka berapa kali perulangan dilakukan?**

• Perulangan dilakukan sebanyak 1 kali. Sifat dasar perulangan DO-WHILE adalah selalu mengeksekusi tubuh perulangan minimal satu kali sebelum memeriksa kondisi.

**2. Sebutkan kondisi berhenti yang digunakan pada perulangan DO-WHILE tersebut!**

• Kondisi berhenti ditentukan oleh pengkondisian 'if (namaPelanggan.equalsIgnoreCase("batal"))' yang mengeksekusi perintah 'break;' untuk keluar dari perulangan.

**3. Apa fungsi dari penggunaan nilai true pada kondisi DO-WHILE?**

• Penggunaan nilai 'true' membuat perulangan bernilai selalu benar secara konstan (infinite loop) sehingga program terus berulang memproses pelanggan berikutnya hingga diperintahkan keluar secara eksplisit melalui perintah 'break'.

**4. Mengapa perulangan DO-WHILE tersebut tetap berjalan meskipun tidak ada komponen inisialisasi dan update?**

• Karena perulangan diatur secara logika sentinel (berdasarkan input kata 'batal' oleh pengguna) dan bukan berbasis angka counter (seperti i++). Selama kata 'batal' tidak diinputkan, kondisi while(true) akan selalu bernilai true.

## Tugas 1: Program Penjualan Tiket Bioskop

### Kode Program (Bioskop_11.java):

```
import java.util.Scanner;

public class Bioskop_11 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        final int HARGA_TIKET = 50000;
        int totalTiketTerjual = 0;
        double totalPenjualanHarian = 0;
        int jumlahTiket;

        while (true) {
```

```
            System.out.print("Masukkan jumlah tiket yang dibeli (ketik -1 untuk selesai): ");
            jumlahTiket = sc.nextInt();

            if (jumlahTiket == -1) {
                break;
            }

            if (jumlahTiket < 0) {
                System.out.println("Jumlah tiket tidak valid (tidak boleh negatif). Silakan masukkan ulang!");
                continue;
            }

            double totalHarga = jumlahTiket * HARGA_TIKET;
            double diskon = 0;

            if (jumlahTiket > 10) {
                diskon = 0.15;
            } else if (jumlahTiket > 4) {
                diskon = 0.10;
            }

            double hargaSetelahDiskon = totalHarga - (totalHarga * diskon);
            totalTiketTerjual += jumlahTiket;
            totalPenjualanHarian += hargaSetelahDiskon;

            System.out.println("Subtotal sebelum diskon : Rp " + totalHarga);
            System.out.println("Diskon                  : " + (int)(diskon * 100) + "%");
            System.out.println("Total bayar transaksi   : Rp " + hargaSetelahDiskon);
            System.out.println("----------------------------------------");
        }

        System.out.println("\n=== REKAP PENJUALAN TIKET HARI INI ===");
        System.out.println("Total tiket terjual  : " + totalTiketTerjual + " tiket");
        System.out.println("Total hasil penjualan: Rp " + totalPenjualanHarian);

        sc.close();
    }
}
```

## Tugas 2: Program Penghitungan Pembayaran Parkir

### Kode Program (Parkir_11.java):

```
import java.util.Scanner;

public class Parkir_11 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int jenis, durasi;
        int total = 0;

        do {
            System.out.print("Masukkan jenis kendaraan (1: Mobil, 2: Motor, 0: Keluar): ");
            jenis = sc.nextInt();

            if (jenis == 1 || jenis == 2) {
                System.out.print("Masukkan durasi parkir (jam): ");
                durasi = sc.nextInt();

                if (durasi > 5) {
                    total += 12500;
                } else {
                    if (jenis == 1) {
                        total += durasi * 3000;
                    } else if (jenis == 2) {
                        total += durasi * 2000;
                    }
                }
            } else if (jenis != 0) {
                System.out.println("Jenis kendaraan tidak valid!");
            }
        } while (jenis != 0);

        System.out.println("\nTotal pembayaran parkir: Rp " + total);

        sc.close();
```

```
    }
}
```